Proyecto de Robótica de Servicios



Controlador mioeléctrico-inercial aplicado a un ABB IRB120 para personas con limitaciones físicas del miembro superior

Memoria técnica

Trabajo hecho por:

Uriel Francisco Lavado Garzón
Juan Pérez Fernández
Andrés Vilches García
Eduardo Gregorio Gómez
Juan Pablo Neme Jérez

ÍNDICE

1. RESUMEN Y DESCRIPCIÓN DE LA APLICACIÓN 2

2. DESCRIPCIÓN DEL SISTEMA DE INTERACCIÓN HUMANO-ROBOT UTILIZADO Y FUNCIONALIDADES 2

2.1 CONTROL MIOELÉCTRICO 3

2.2 CONTROL INERCIAL 4

3. MANUAL DE DESPLIEGUE Y FUNCIONAMIENTO DE LA APLICACIÓN 5

3.1 SIMULACIÓN 7

3.2 MONTAJE REAL 8

3.3 INTERFAZ GRÁFICA 9

4. DESCRIPCIÓN DE LAS ACTIVIDADES REALIZADAS EN EL ROBOT 10

5. CONCLUSIONES Y POSIBLES MEJORAS 11

6. BIBLIOGRAFÍA 12

1. RESUMEN Y DESCRIPCIÓN DE LA APLICACIÓN



Figura 1 . Imagen del robot ABB IRB120.

El proyecto consiste en el control de un manipulador ABB IRB120 mediante control mioeléctrico para la posición y control inercial para la orientación.

La inspiración del proyecto parte de que una persona con alguna limitación física en el miembro superior pueda ser capaz de controlar un brazo robótico sin necesidad de utilizar las manos utilizando métodos alternativos vistos en clase.

Un ejemplo de nuestro proyecto incorporado a una aplicación real sería la implementación de nuestro controlador a un brazo robótico auxiliar en sillas de ruedas inteligentes para personas con movilidad reducida. Por supuesto, un robot ABB IRB120 es un manipulador de tipo industrial que no sería adecuado para este caso. Sin embargo, la adaptación del controlador a un robot colaborativo sí que podría utilizarse sin modificar la estructura principal del programa.

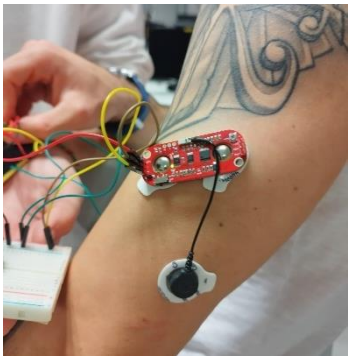
Por hacer un resumen, con este proyecto se busca la inclusividad de las personas discapacitadas para mejorar su calidad de vida a través de un controlador robótico adaptado a sus limitaciones. Dentro del campo de la robótica de servicios, este proyecto se clasificaría dentro del campo de la robótica asistiva; ya que ofrece asistencia a personas con algún grado de dependencia para mejorar su calidad de vida.

2. DESCRIPCIÓN DEL SISTEMA DE INTERACCIÓN HUMANO-ROBOT UTILIZADO Y FUNCIONALIDADES

Como se ha comentado en el apartado anterior, el controlador robótico puede dividirse en un controlador de posición mediante control mioeléctrico y en un controlador de orientación mediante control inercial. En los siguientes subapartados se describe lo anterior con mayores niveles de detalles.

2.1 CONTROL MIOELÉCTRICO





Figuras 2 y 3. Modelo utilizado del sensor mioeléctrico de MyoWare y colocación en el brazo

El control mioeléctrico se encarga de controlar la posición del robot mediante los impulsos eléctricos del grupo muscular del brazo. Debido a que solamente se dispone de dos sensores mioeléctricos, solamente se podrá determinar cuatro estados posibles: reposo, flexión, extensión y co-contracción.

De entre todos los posibles controladores mioeléctricos vistos en clase, se decide optar por un control articular mediante una máquina de estados finita para las tres primeras articulaciones del robot.

El reposo hará que la articulación no se mueva; la flexión hará que la articulación gire a velocidad constante en sentido antihorario; la extensión hará lo mismo que la flexión, pero en sentido horario; y la co-contracción hará que se cambie a la articulación siguiente. Los músculos involucrados son el tríceps y el bíceps.

Los umbrales de activación, tanto para el canal de flexión como el de extensión, son de 2.0 V, determinados mediante prueba y error durante la experimentación.

El sensor mioeléctrico utilizado es el MyoWare Muscle Sensor v1.0 de Advancer Technologies. Su principal característica es que tiene un preprocesamiento de la señal que permite extraer la envolvente de la señal de forma directa.

2.2 CONTROL INERCIAL

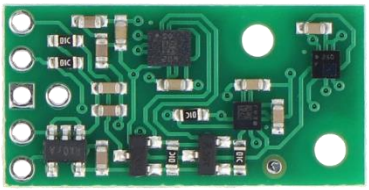


Figura 4. Modelo utilizado del sensor inercial

El control inercial se encarga de controlar la orientación del robot mediante la lectura y posterior codificación de la posición del sensor. A diferencia del sensor mioeléctrico, éste no necesita una máquina de estados, ya que se encarga de controlar las dos últimas articulaciones al mismo tiempo.

El sensor inercial utilizado es el AltIMU-10 v6 de Pololu Robotics and Electronics. Este sensor está equipado con el giroscopio y acelerómetro en 3 ejes LSM6DSO, el magnetómetro en 3 ejes LIS3MDL y el barómetro digital LPS22DF. Mediante todos ellos, el dispositivo es capaz de funcionar como una unidad de medición inercial (IMU) y como un altímetro.

Para llevar a cabo el cometido de esta práctica, ha sido utilizado el LSM6DSO para poder medir el roll y el pitch. Gracias a esto, ha sido posible sacar los ángulos de inclinación a los que se sometía el IMU, los cuales se han codificado de forma que, si giramos en el sentido horario respecto al roll, la articulación 4 girará en sentido horario y al contrario si el sensor detecta que el giro es en un sentido antihorario. Por otra parte, si detecta un giro horario en el roll, se realizará un giro en sentido horario en la articulación 5 o al revés en caso de detectar el giro contrario.

3. MANUAL DE DESPLIEGUE Y FUNCIONAMIENTO DE LA APLICACIÓN

Figura 5. Bróker MQTT para el envío de datos

HiveMQ es un bróker web gratuito para el envío de datos MQTT, ya sea mediante un archivo CSV en el caso de que se hiciese por simulación, o bien a partir de datos reales extraídos de los sensores mioeléctrico e inercial en el microcontrolador ESP32 a través de los topics "**/emg/flex**" y "**/emg/ext**" a una velocidad de frecuencia de unos 4-5 Hz, aproximadamente.

Su configuración es sencilla y amigable. Sin embargo, como a nivel de usuario no se necesita conocer el usuario y la contraseña para enviar los datos (y por lo tanto, para utilizar nuestra aplicación); ya que se presupone que se tiene el programa previamente cargado en la ESP32, no nos vamos a esmerarnos más en detalles técnicos sobre su configuración.

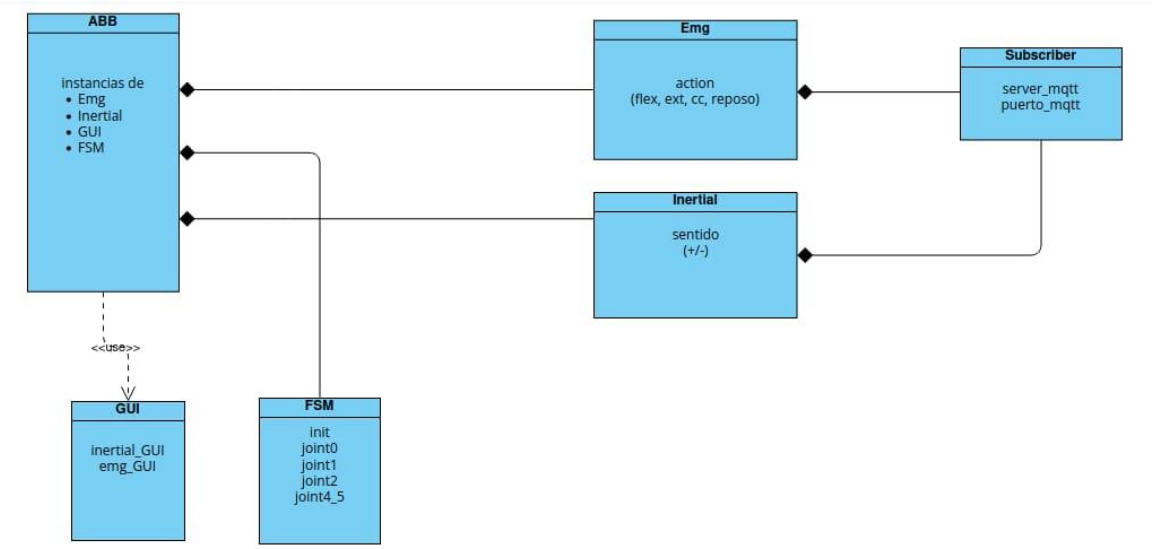


Figura 6. Diagrama UML de las distintas clases del proyecto

Hablando más en detalle sobre las clases que se utilizan para que funcione todo el proyecto, se anexa la **Figura 6** que nos permite ver la relación entre las distintas clases. A continuación, se explica brevemente qué realiza cada clase:

La clase **Subscriber** se encarga de suscribirse a los topics del sensor EMG y a los del sensor IMU, para leer los valores que llegan por dichos topics.

La clase **EMG** se encarga de decodificar los valores de los topics de flexión y extensión para saber qué acción se está realizando (reposo, flexión, extensión o co-contracción). También tienen una instancia de la clase **Subscriber.**

La clase **IMU**, al igual que la clase **EMG**, se encarga de decodificar los valores enviados por sus topics específicos para determinar la acción que se está realizando. También tienen una instancia de la clase **Subscriber.**

La clase **FSM** se encarga de crear la máquina de estados con sus nodos y transiciones necesarias para el correcto funcionamiento del controlador mioeléctrico-inercial. Se detalla más abajo, a partir de la **Figura 7**.

La clase **GUI** se encarga de llevar a cabo la interfaz gráfica del programa para que sea amigable al usuario y del que se detallará en el **Apartado 3.3** .

Y finalmente, la clase **ABB**, que engloba instancia de las clases **EMG**, **Inertial**, **GUI** y **FSM,** y es la principal encargada de gestionar todo el funcionamiento del programa.

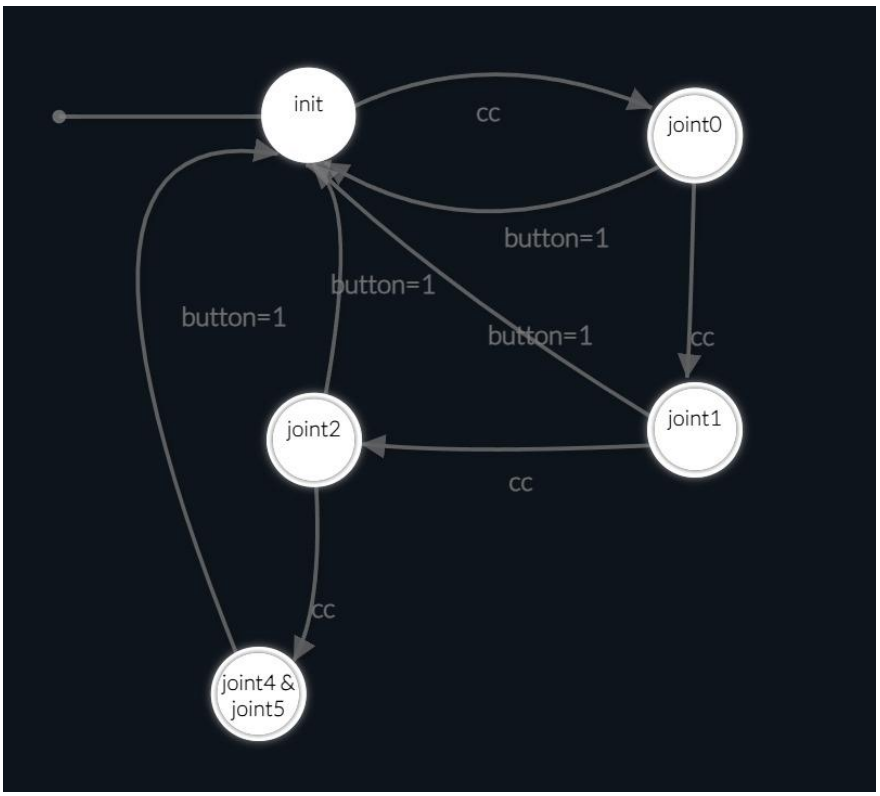


Figura 7. Máquina de estados de la aplicación completa

Sabiendo ya cómo se realiza el envío de datos y cómo se relaciona las distintas clases utilizadas en el proyecto, sería necesario saber cómo funciona la máquina de estados que controla al robot.

Como se aprecia en la **Figura 7**, se parte de un estado inicial en el que no se controla ninguna articulación, por lo que el ABB IRB120 no se moverá salvo que se realice una co-contracción en el sensor mioeléctrico.

El cambio de estado entre joints se produce gracias a la co-contracción; ya que permanecer en reposo, extensión o flexión solamente hará que la articulación se quede quieta o se mueva en sentido horario o antihorario, respectivamente.

Como se comentó en el **Apartado 2.1**, los tres primeros joints se controlan individualmente para la posición del manipulador; mientras que solamente se necesita un estado para los joints 4 y 5, que se encargan de la orientación del robot como ya se dijo en el **Apartado 2.2**.

Además del funcionamiento normal del programa, se incluye un botón de ESP32 de emergencia que devuelve la máquina de estados a su estado inicial, independientemente de la articulación en la que se encuentre.

3.1 SIMULACIÓN

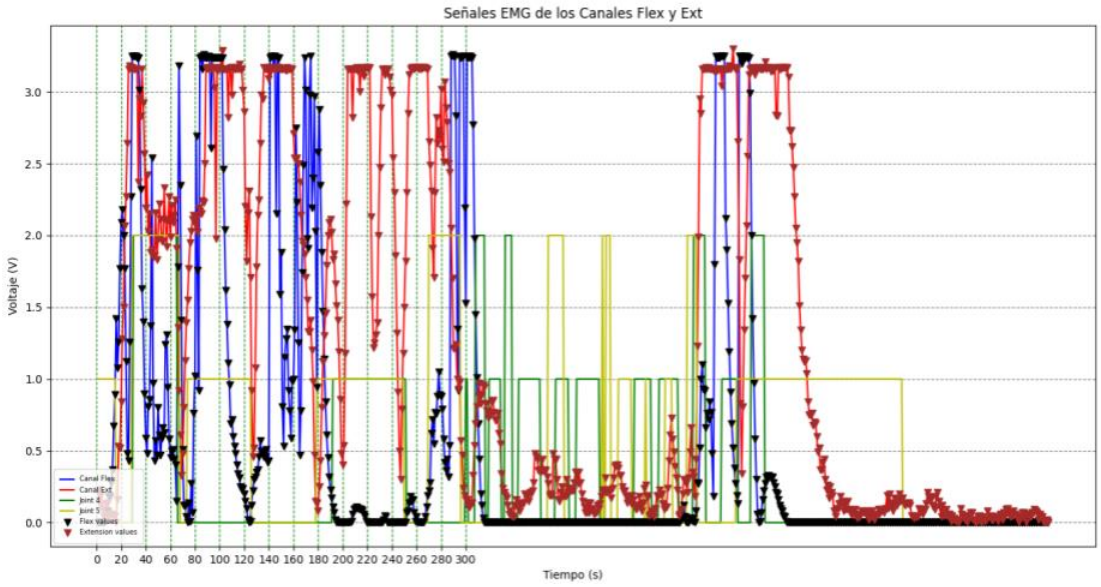


Figura 8. Gráfica del archivo CSV utilizado durante la simulación

La leyenda del archivo CSV de la **Figura 8** se puede resumir rápidamente:

La gráfica azul representa los datos de extensión, mientras que la gráfica roja representa los datos de flexión. A su vez, los triángulos negros y rojos representan los valores exactos de las lecturas de los canales de extensión y de flexión, respectivamente.

Las gráficas verde y amarilla representan los datos del sensor inercial para los joints 4-5.

La serie de pasos para ponerlo en marcha se detalla a continuación:

1. Abrir **readCSV_and_sendMQTT.py** teniendo el **csv** que se ha hecho anteriormente en clase

2. Iniciar Gazebo con este comando: **$ roslaunch abb_irb120_gazebo irb120_3_58_gazebo.launch**

3. Una vez abierto Gazebo, se abre Rviz con este comando: **$ roslaunch abb_irb120_moveit_config moveit_planning_execution_gazebo.launch**

4. Una vez se mandan los datos se puede ejecutar **ABB.py** con el Gazebo y el RVIZ cargados previamente del paso anterior.

5. Ya por último se ejecuta el **GUI_v3.py** y todo listo.

3.2 MONTAJE REAL

Listado de conexiones del controlador mioeléctrico-inercial		
Pin asociado	**Sensor utilizado**	**Número de pin**
SDA	IMU	21
SCL	IMU	23
GND	IMU	GND global
VIN	IMU	VCC (3,3V)
Analógico	EMG	32
Analógico	EMG	33
GND	EMG	GND global
VIN	EMG	VIN global

Tabla 1. Listado de los pines asociados al esquema eléctrico utilizados en el montaje real.

Como se quiere controlar tanto la flexión como la extensión, se necesitará dos sensores mioeléctricos (también llamados EMG) en vez de uno sólo. Por eso, se tiene dos pines analógicos en la **Tabla 1** apuntando a los pines 32 y 33.

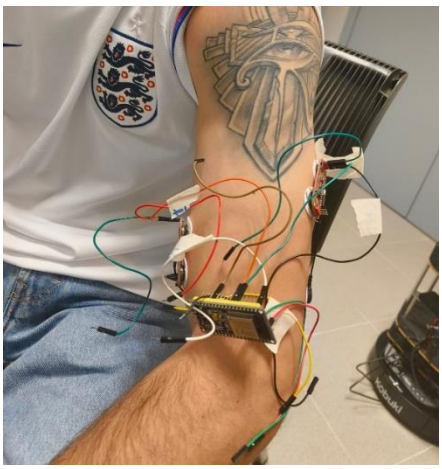


Figura 9. Montaje real sobre el ESP32 a partir de la **Tabla 1**.

Las desventajas del montaje real fueron que algunos cables podían soltarse debido a movimientos bruscos del brazo y que, en general, era bastante incómodo de llevar quién fuese el portador del prototipo.

La serie de pasos cambia ligeramente con respecto a la serie de pasos que se hacía para la simulación, que se detalla a continuación:

1. Se conecta el ESP32 con los componentes que se van a utilizar (EMGs, IMU y botón) como se indica en la **Tabla 1** y se muestra en la **Figura 9**.

2. Se lanza el código **imuAndemg_mqtt.ino** que procesa los datos extraídos de los sensores y los envía mediante MQTT al bróker.

3. Repetir a partir del **Paso 2** de la serie de pasos de la página anterior, que sería los pasos para iniciar Gazebo, Rviz, **ABB.py** y **GUI_v3.py,** en ese orden.

3.3 INTERFAZ GRÁFICA

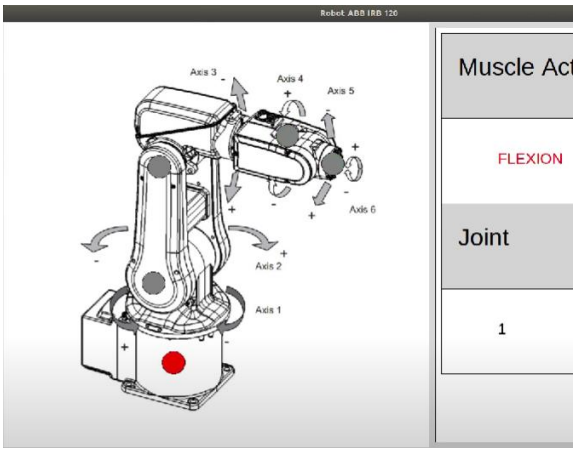


Figura 10. Vista de la interfaz gráfica

La interfaz gráfica se encarga de ofrecer una visualización amigable al usuario del funcionamiento de la aplicación según los comandos de control que se estén enviando mediante el controlador mioeléctrico-inercial.

Como se aprecia en la **Figura 10**, la parte de la izquierda nos muestra una imagen del ABB IRB120 con la numeración de los joints y unos LEDs en cada uno de ellos que representan el sentido de rotación de estos.

El código de colores es sencillo de comprender:

- Un LED que esté apagado (es decir, de color gris) puede estarlo por dos motivos distintos:

  1. Que no sea la articulación que se esté controlando actualmente.

  2. Que sea la articulación que se esté controlando actualmente, pero el comando de control sea "reposo" o "co-contracción".

- Un LED que esté de color rojo significa que la articulación está girando en sentido horario o que está llegando una señal de flexión.

- Un LED que esté de color verde significa que la articulación está girando en sentido antihorario o que está llegando una señal de extensión.

En la parte de la derecha, tenemos dos valores que nos muestra el joint que se está controlando actualmente (Joint 1 – Joint 5) y la acción que se está enviando actualmente (Reposo, Flexión, Extensión y Co-Contracción).

Como podemos ver, la interfaz es sencilla de comprender, con un diseño básico pero funcional y suficiente para nuestra aplicación.

4. DESCRIPCIÓN DE LAS ACTIVIDADES REALIZADAS EN EL ROBOT

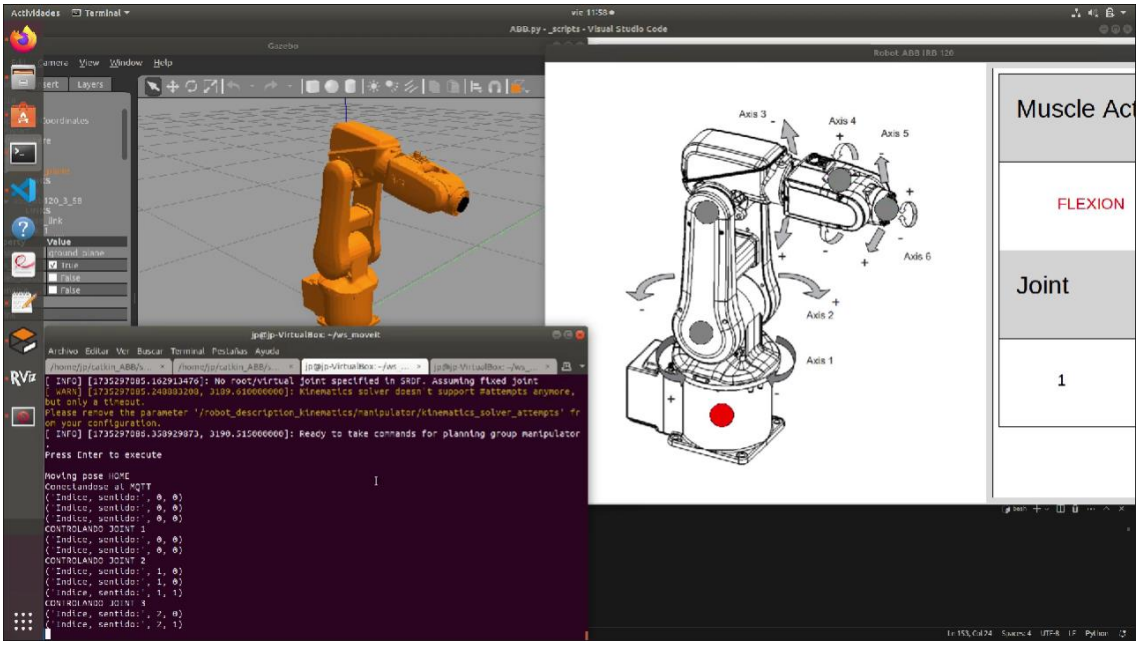


Figura 11. Imagen genérica para ocupar espacio en el Word

Como ya se indicó en los apartados anteriores, el objetivo de nuestra aplicación es el control de un robot manipulador ABB IRB120 orientado para personas con limitaciones físicas del miembro superior.

Las actividades realizadas en el robot, tanto en simulación como en la vida real, se trata de poder controlar la posición, la orientación del robot y de poder incorporar un botón de emergencia en el ESP32 para poder volver al estado inicial de la máquina de estados tras un fallo.

Los vídeos que demuestran su funcionamiento, aparte de la demostración presencial a José Luis en clase de prácticas, se pueden consultar en el enlace de Youtube que estará subido en la bibliografía.

ADVERTENCIA: En el vídeo, la GUI (es decir, la interfaz gráfica) y el ABB real no van perfectamente sincronizados por el tema de las comunicaciones, pero si se integrase la GUI sobre el mismo archivo de control del ABB (en vez de hacerlo en un script en paralelo) funcionaría correctamente.

5. CONCLUSIONES Y POSIBLES MEJORAS

En conclusión, este proyecto ha supuesto un arduo trabajo que ha llevado muchas dificultades de por medio y que ha imposibilitado algunos objetivos que se habían planteado inicialmente.

Posiblemente, el mayor objetivo que no se ha podido cumplir ha sido probar el programa sobre el ABB real debido a muchos errores a la hora de configurarlo correctamente. Sin embargo, eso no ha sido impedimento para probarlo en simulación mediante Gazebo, ya sea mediante un archivo CSV o tomando datos reales de los sensores. No obstante, el funcionamiento ha estado alejado de lo ideal debido a múltiples incidencias como la inestabilidad de la red Wi-Fi utilizada y de la velocidad de transmisión del protocolo MQTT, que ha mermado la frecuencia de funcionamiento ideal del programa por debajo de los 4 Hz. Otra incidencia que destacar sería la dificultad del control de los sensores mioeléctricos debido a su colocación en el tríceps y en el bíceps, ya que esto dificultaba realizar flexiones y extensiones aisladas sin un entrenamiento previo. Por último, a nivel de la GUI, los datos mostrados en la interfaz gráfica iban descompasados con respecto a los que se mostraba en la terminal o en Gazebo, como ya se comentó anteriormente.

Sin embargo, no todo ha sido malas noticias; ya que se pudo comprobar el correcto funcionamiento de los sensores por separado, así como el de una demostración funcional del prototipo completo durante la clase de prácticas a pesar de que no iba del todo bien.

No obstante, si se pudiera explorar otras vías de comunicación más eficientes para el tema de las comunicaciones; así como el de construir un prototipo más cómodo y con una mejor colocación de los sensores, se podría haber aumentado la frecuencia por encima de los 4 Hz y conseguir un funcionamiento más estable del programa.

Las posibles mejoras del controlador mioeléctrico-inercial podrían resumirse en seis puntos principales que se enumeran a continuación:

A. Aumento de la frecuencia de funcionamiento del controlador por encima de los 10 Hz para un funcionamiento de la aplicación más fluido. Sin embargo, establecer una frecuencia demasiado alta (>100Hz) también puede ser problemático por la redundancia de datos similares y por la presencia de comportamientos inesperados en el programa debido a ello.

B. Un controlador mioeléctrico-inercial más complejo en cuánto a funcionamiento. Por ejemplo, que las velocidades angulares de los ejes en el control mioeléctrico sea proporcional a la extensión/flexión o que haya mejoras en el filtro del control inercial para reducir la sensibilidad de los datos en la orientación del efector final del robot.

C. Una estrategia más compleja en el control mioeléctrico como, por ejemplo, modificar la máquina de estados para que el control en posición sea de tipo cartesiano (un estado podría controlar el eje "x", otro el eje "y" y otro el eje "z"). Sin embargo, se nos presentaría nuevos retos como la solución en tiempo real de la cinemática inversa o sobre la gestión de las singularidades del robot en una determinada trayectoria cartesiana.

D. Incluir funcionalidades adicionales al robot aparte del control en posición y en orientación como, por ejemplo, añadir una pinza para la manipulación de objetos y otras tareas que requieran de ello como abrir y cerrar puertas.

E. Un proceso de calibración previa en función del usuario para el control mioeléctrico para determinar los umbrales de activación de manera automática.

F. Un estudio sobre el funcionamiento del controlador mioeléctrico-inercial y de la interfaz de usuario con diversos sujetos reales para conocer en qué puntos se debe de mejorar el programa principal y también en la interfaz gráfica.

6. BIBLIOGRAFÍA

Repositorio de Github del proyecto:

https://github.com/JuanPerez07/Proyecto_Servicios

Vídeo de Youtube del funcionamiento del programa en simulación:

https://youtu.be/jI7ZveMHEAw

Página del sensor mioeléctrico utilizado en el proyecto:

https://ultra-lab.net/producto/myoware-muscle-sensor-sparkfun-sen-13723/

Página del sensor inercial utilizado en el proyecto:

https://www.pololu.com/product/2863